Classifying Opinions and Facts Using Traditional NLP Methods and Large Language Models

Matteo Urthaler and Markus Zangl

Graz University of Technology, Graz, Austria

June 25, 2026

Abstract

In this research paper, a two-stage text classification pipeline for distinguishing between facts and opinions is presented. The first stage involves the extraction of linguistic features, including part-of-speech ratios, named entities, readability metrics and sentiment. These features are used to train a traditional machine learning classifier. The second stage analyzes generative text classification using Large Language Models (LLMs), namely Zero-Shot, Few-Shot, Chain-of-Thought and Retrieval-Augmented Generation. Both stages are using a data preprocessing pipeline to filter corrupt data and prompt injections. The results of a 5-fold-cross-validation loop show that an optimized Linear Support Vector Machine of stage 1 yields a mean accuracy of 80.54%. A hybrid model of stage 2 comprising of carefully chosen Few-Shot examples and Chain-of-Thought prompts outperformed all standalone methods with an mean accuracy of 85.03%. Retrieval-Augmented Generation configurations yield lower accuracies compared to the other models due to the incorporated retrieval noise. We observed that using carefully engineered features improves the performance of classic models, while using hybrid prompts maximizes LLM reasoning ability despite having some operational bias to classify uncertain texts as facts.

1 Introduction

Distinguishing between objective facts and subjective opinions is a foundational task in Natural Language Processing (NLP). The main goal of this work is to implement a text classification pipeline that decides whether a given text is either a fact or an opinion.

Our work is split up into two stages. Both stages aim to achieve the goal, but in two different ways. Stage 1 focuses on traditional NLP techniques and classic machine learning models. We build a simple knowledge base by extracting linguistic features from labeled texts and train a classifier with it. Stage 2 focuses on modern Large Language Models to explore generative text classification possibilities. We test different advanced prompting methods and evaluate them.

2 Related Work

Before the development of the larger models, text processing using neural networks relied on recurrent structures which process text sequentially. As explained by Olah (2015), understanding context and word order was revolutionized by Long Short-Term Memory networks. LSTM networks became the standard.

The self-attention mechanism and the Transformer architecture by Vaswani et al. (2017) have transformed the field. The architecture eliminated the need for recurrent loops, enabling models to process entire sentences simultaneously. Large Language Models (LLMs)

are built on Transformers. But modern LLMs have serious limitations, such as generating wrong information or hallucinating. Recent work by Anthropic AI (2024) underlines the importance of keeping an eye on how language models organize their internal reasoning. By prompting the model to look at its own intermediate thoughts, just like we do with Chain-of-Thought prompting, we can better understand how a model decides whether a statement is a fact or an opinion.

3 Materials and Methods

This section explains the materials and analytical methods used for both stages.

3.1 Data Preprocessing Pipeline (Stage 1 and 2)

Raw text data often has encoding issues, formatting errors or in our case text typos and wrong characters that can destroy a robust text classification pipeline. The provided datasets **dataset.csv** and **validationset.csv** were already relatively clean. We still identified several inconsistencies by manually inspecting the data. We built a data cleaning script to fix text and to flag bad rows before looking at features or using it for the LLM stage. More detailed Information about used programming language, libraries and other technologies can be found in Appendix: Implementation Note. We clean characters using the Unicode NFKC standard and remove extra spaces. Then we use three filters to flag bad rows:

- **Other Language Text:** We flag text with Chinese, Japanese or Korean signs to keep out dataset in English.
- **Corrupt Text:** We count occurring symbols in the text. If more than 30% of the characters are symbols, we flag the row.
- **Prompt Injections:** We search the text for possible AI instructions, like ”system prompt” or ”note to ai:” and flag these rows as well.

Flagged rows will get a final prediction of ”none”.

3.2 Feature Engineering (Stage 1)

To be able to distinguish between facts and opinions we extracted 25 features across the five hinted main areas:

1. **POS Tagging:** We count nouns, verbs, adjectives, adverbs, pronouns, and proper nouns. These counts are then divided by the total number of words to get their corresponding ratios. We also calculate the ratio of adjectives to nouns.
2. **Named Entity Recognition:** We count total named entities, people, organizations, geo-political entities, locations, and dates. These counts are then also divided by the total number of words to get their corresponding ratios.
3. **Emotion Analysis:** We use a subjectivity score and measure how neutral the text is by looking at compound sentiment, positive sentiment, negative sentiment and neutral sentiment.
4. **Readability Analysis:** We use the average sentence length, average syllables per word and came up with a difficult words ratio by counting words with more than 3 syllables divided by total amount of words and a scaled Flesch Reading Ease score.
5. **Error Analysis:** We count spelling and grammar mistakes and divide them by the total number of words. Additionally we added a total error ratio where spelling and grammar errors were combined.

Not all of those features were used in the final pipeline, more information about why certain features were canceled can be found in the Appendix: Approaches tried.

3.3 Euclidean Distance Baselines (Stage 1)

To create a baseline, we calculate the average feature vector for facts and opinions for every hinted main area. The goal of these baselines was to get a feeling if our features actually work, so we created guesser scripts for every area where for every text the Euclidean distance

to both averages is calculated and the closer one is picked. The baseline results can be seen in the section Results: Stage 1 Performance Metrics.

Based on these baseline results and additional tests conducted while running the classifiers, a new set of 17 features was selected from the original 25 features. More information about that can be seen in the Appendix: Approaches tried.

3.4 Machine Learning Pipeline (Stage 1)

We selected the top 9 features that worked best while testing for the best ML model, these 9 features were: noun_ratio (POS), adverb_ratio (POS), pronoun_ratio (POS), propernouns_ratio (POS), entity_ratio (NER), date_ratio (NER), sentiment_neu (emotion), subjectivity (emotion), and flesch_reading_ease (readability).

We tested several models. Random Forest got around 70% accuracy, Logistic Regression got around 75% accuracy. A Linear Support Vector Machine with performed the best, reaching approximately 79% accuracy. After finding the best model architecture in general, we continued testing different hyper-parameters to improve the pipeline performance even more. The result of the final Linear SVM model can be seen in section Results. More information about tested Model architectures and hyperparameter tuning test can be found in the Appendix: ML Model Testing.

3.5 LLM-Based Text Classification (Stage 2)

In stage 2 the goal was to classify the text documents with the use of LLMs. We have tried several approaches and determined their accuracy based on the provided dataset. The two best approaches were then merged to further increase accuracy. The tested approaches are: Zero-Shot Learning, Few-Shot Learning, Chain-of-Thought(CoT) Prompting and Retrieval-Augmented Generation (RAG). Each of the approaches was implemented and tested, the results were documented, and compared to make an educated choice.

3.6 Implemented Approaches (Stage 2)

These are the approaches that we chose to implement and test:

1. Zero-Shot Learning: This is done by instructing the LLM to classify the text directly via a carefully engineered system prompt without providing any prior examples. We did this with two different LLMs. First, with bart-large-mnli, supplying only the candidate_labels and then with SmolLM2-1.7B-Instruct, this time with a prompt. The prompt was designed with the help of a LLM. This was a deliberate choice, since LLMs have an understanding of their own input preferences the generated prompt makes it convey the required functionalities much more precisely. Of coures specifications were specifically humanly requested. The results were about 60% for both LLMs, SmolLM2 was slighty better.

2. Few-Shot-Learning: This is done by providing the LLM with 2 carefully curated examples of facts and opinions each within the prompt context to guide its reasoning pattern. Here too SmolLM2-1.7B-Instruct was chosen as the LLM. For the example two different styles were used. First, examples generated by an LLM and then also handpocked examples from the dataset. As prompt the same as in the Zero-Shot Learning was used. For the generated examples the accuracy turned out to be at about 75% and for the Handpicked ones an accuracy was around 82%, showing a significant increase in accuracy.

3. Chain-of-Thought(CoT) Prompting: This is done by forcing the model to explain its reasoning step-by-step before outputting the final classification, which is particularly useful for untangling complex, mixed statements. Again SmolLM2-1.7B-Instruct was used for this task with some improvements to the prior prompt. This time the prompt requests an argument of three sentences explaining why the classification is

correct. The tests resulted in an accuracy of about 78% for this approach.

4. Retrieval-Augmented Generation (RAG): This is done by using an external search tool to verify claims and retrieve context about the text before asking the LLM to classify it. Again SmolLM2-1.7B-Instruct was used as the LLM and the external search was done with DuckDuckGo from langchain_community.tools. Via the external seach additional information is retrieved to classify the claims from the text. This approach landed at around 58%.

Not all of these approaches were used in the final pipeline. More information about why certain approaches were canceled can be found in the Appendix: Approaches tried.

3.7 Combined Approaches (Stage 2)

From the implemented approaches two were selected for further optimization by combination.

1. Retrieval-Augmented Generation (RAG) and Chain-of-Thought(CoT): The basic idea is to add the reasoning factor from CoT to the external knowledge aspect from RAG. Ideally with this A strong argument could be made for the classification. The combined results had an accuracy of 71%.

2. Chain-of-Thought(CoT) and Few-Shot-Learning: The two strongest candidates from our implemented approaches combine the element of reasoning with the provided examples. Together this forms a strong pipeline in which the model first solidifies its understanding through the examples and then provides a classification with the intermediate reasoning steps. The combined results had an accuracy of about 85%.

4 Results

4.1 Stage 1 Performance Metrics

To check the performance of our guessers and thus their underlying feature quality, we used a Stratified 5-Fold Cross-Validation loop ($K = 5$). The average results for each baseline group are shown in Table 1.

Table 1: 5-Fold Cross-Validation performance of baseline guessers.

Feature Domain	Mean Acc. (%)	Std. Dev. (σ)
NER Baseline	70.00	12.46
POS Baseline	66.80	7.22
Emotion Baseline	65.80	5.38
Readability Baseline	58.60	4.32
Error Baseline	50.40	10.31

Our final Linear SVM model used the combined 9 features. We ran it through the same 5-Fold Cross-Validation loop, and it achieved a **Mean Accuracy of 80.54%**. This shows that combining the right features gives much better results than using just one group.

4.2 Stage 2 Performance Metrics

To check the performance of our LLM based models, we used the provided dataset and ran the model on the entirety of the content. We calculate the Accuracy for a general overview and precision, recall and the confusion matrix for preciser results. The average results for each are shown in Table 2.

Table 2: Performance of all the implemented baseline approaches

Approach	Accuracy (%)
Few-Shot Handpicked	81.87
CoT Prompting	78.36
Few-Shot Generated	74.85
Zero-Shot SmolLM2	61.40
Zero-Shot BART	59.53
RAG	57.89

The same was done to achieve the accuracy for the two combined models. The results are illustrated in table 3.

Table 3: Performance of combined approaches

Combined approaches	Accuracy (%)
Few-Shot + CoT	85.03
CoT + RAG	71.00

As seen in Figure 1 the confusion matrix shows the model correctly classified almost the entirety of the facts and about three quarters of the opinions. This shows the model leans towards fact when uncertain.

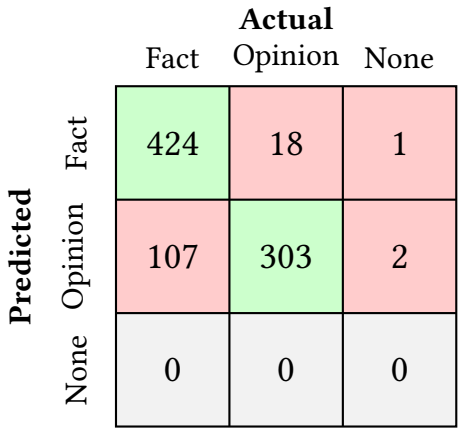


Figure 1: Confusion matrix for the Few-Shot + Chain-of-Thought classifier (accuracy: 85.03%).

5 Discussion

The results show a clean progression throughout the methods. The linear SVM model combined all features effectively and outperformed every single feature domain with an accuracy of 80.54%. This confirms that single features do not suffice for this kind of classification they rather require a combination of semantic, stylistic and syntactic measures.

In stage 2 Few-Shot + Chain-of-Thought achieve the best overall rating with an accuracy of 85.03%, showing that the combination of different LLM approaches can render better results than just the approach by itself. The confusion matrix confirms the bias of the model, it is far more likely to label an opinion as a fact than vice versa.

Chain-of-Thought + Retrieval-Augmented Generation on the other hand underperformed with an accuracy rating of just 71.00%. The retrieval has introduced noise instead of clearer thoughts trough useful context. The search engine result might just not be able to resolve the conflict of opinion or fact. The baseline though confirms that classification without context or some improvement yields similar results as a coin toss.

6 Conclusion

This paper investigates text classification in opinion and and fact. First by using traditional NLP approaches and machine learning and then also LLM based strategies. The linear SVM provided a strong baseline of 80.54% accuracy for the later LLM based Few-Shot + Chain-of-Thought setup that peaked at 85.03% making it the best overall performer. This shows that prompting with explicit reasoning steps can outperform classical feature based approaches. Linear SVM was even outperformed by the handpicked Few-Shot strategy furthermore showing that uncombined LLM approaches can also be stronger. The Zero-Shot approaches have proven to be insufficient and RAG on its own along with in combination as well.

Future work could include exploring additional approaches like Fine-Tuning, Self-Consistency / Multi-Agent Debate or Soft-Prompting / Prompt Tuning. Experimentation with different models or even just finetuning the approaches to handle text that is right on the line of fact and opinion could have a hugh impact as well.

Limitations

Our traditional feature pipeline from stage 1 only works for english texts because it uses english tokenizers and grammatical rules. It cannot understand complex writing styles, such as sarcasm where an opinion is written to appear as a fact.

Ethical Considerations

Automated tools can learn human biases that exist in the training data. If an opinion is mistaken for a fact it could add credibility to fake information, and if a fact is labelled an opinion it could lead people to disregard actual information. Large Language Models can raise sustainability concerns, as training and running these models need huge amount of computational resources and energy. This can lead to large CO2 emissions, especially for large scale models. The environmental impact of using LLM based methods should also be considered.

Acknowledgments

We want to thank the whole teaching team for the clear yet flexible project description and for providing the project templates, which were very helpful throughout the project. We also appreciate the teams professional and constructive responses to questions in the forum, which greatly supported us during the project.

Contributions

Table 4: List of contributions per team member.

Team Member	Contribution
Matteo Urthaler	worked on both stages and report, mainly implemented Preprocessing Pipeline; stage 1 POS Tagging, Named Entity Recognition and Error Analysis, K-Fold testing environment; stage 2 Zero-Shot Learning, Few-Shot Learning; Refactoring and final whole pipeline scripts
Markus Zangl	worked on both stages and report, mainly implemented stage 1 Emotion Analysis, Readability Analysis and machine learning model pipeline; stage 2 Chain-of-Thought Prompting, Retrieved-Augmented Generation and Few-Shot & Chain-of-Thought Hybrid; Refactoring

References

Anthropic AI. Tracing the thoughts of a large language model. https://www.anthropic.com/research/tracing-thoughts-language-model, 2024.

Chris Olah. Understanding lstm networks. https://colah.github.io/posts/2015-08-Understanding-LSTMs/, 2015.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

A Appendix - Overview

In the appendix we share detailed information about the implementation in general like used programming language and frameworks. We also share more information about what did not work out and what we did about it in the development phase, this can be found under Approaches tried. Coming up with a capable ML classifier (no transformer) was a big part of stage 1, therefore the most important tested model architectures and hyper-parameters are presented in ML Model Testing. Last but not least we also want to share how we used LLMs as general support for our work in Usage of AI.

B Appendix - Implementation Notes

The entire project pipeline was built using Python. We used several frameworks and open-source libraries for preprocessing, feature extraction and ML model training:

- **pandas:** loading, cleaning, manipulating datasets.
- **spaCy (en_core_web_sm):** tokenizing text, extracting POS tags, finding named entities.
- **TextBlob:** calculating absolute text subjectivity score.
- **NLTK (VADER Lexicon):** measuring neutral sentiment ratio.
- **textstat:** calculating the Flesch Reading Ease score and find difficult words.
- **language_tool_python:** counting grammatical and spelling errors.
- **scikit-learn:** training, tuning and evaluating machine learning models

C Appendix - Approaches tried

This section explains the methods and configuration choices we tried but changed or removed during development of both stages.

C.1 Stage 1

We started with a set of 25 features to ensure no possibly useful information could be missing. But using all of them introduced noise into our guessers and ML models, rather than being useful. We performed two rounds of feature reduction, simply because we saw at each stage in the project that the guesser or model accuracies were getting better with less features.

1. **Round 1 (from 25 to 17 features):** This happened when we tried optimizing the performance of our 5 guessers (POS Tagging, NER, Emotion Anlaysis, Readability Anlaysis, Error Anlaysis). We found that positive, negative and compound sentiment scores created bias, so we removed them. We also removed raw sentence lengths and syllables because they were already captured by the combined Flesch Reading Ease Score. The features from error analysis were also completely removed because the error baseline guesser was basically a coin toss machine. The removal of these features lead to performance increase in the corresponding guessers.
2. **Round 2 (from 17 to 9 features):** This happened after we found a pleasing ML model and wanted to tune it. We looked at feature weights inside the model and found out that a set of 9 features were really impacting the model and the other ones were basically noise. This reduction boosted our final model mean accuracy to over 80%.

C.2 Stage 2

Development of stage 2 included several different strategies that were refined to come to the final configuration.

1. Prompts: Initially prompts were completely written by hand which produced worse results because of the lack of linguistic precision. Hand written prompts were either too broad or too strict causing the model to misclassify edge cases and therefore yielding worse results. Of course the generated prompts have then been manually overlooked and tuned.

2. Runtime: We tried running our scripts locally on our own hardware, which (although we used quite small models) was far to slow due to old hardware (or not having a good GPU at all). We proceeded to run our scripts with Google Colab T4 GPU's.

D Appendix - ML Model Testing

Several traditional machine learning architectures were tested on our labeled dataset to find the best model for the classification task. The following architectures were tested:

- **Support Vector Machine (Linear Kernel):** We tested different values for the regularization parameter C. Setting $C = 1.0$ gave us our highest test split accuracy of 80% (80.54% mean accuracy under full 5-fold cross-validation). Changing C to 1.1 or 1.2 dropped the accuracy to 78%, while dropping C to 0.5 resulted in 77% accuracy.

- **Support Vector Machine (Polynomial Kernel):** Using a polynomial kernel of degree 2 achieved a lower accuracy of 77%.

- **Support Vector Machine (Sigmoid Kernel):** The sigmoid kernel performed very poorly on our feature space, achieving only 54% accuracy.

- **Logistic Regression:** This model provided a solid baseline with an accuracy of 75%.

- **Random Forest Classifier:** We tested a Random Forest configuration with 150 estimators and a maximum tree depth of 10. It performed worse than the linear models, achieving an accuracy of 70%.

Based on these testing results, the Linear SVM architecture with $C = 1.0$ was selected as our final Stage 1 model.

E Appendix - Usage of AI

During this project we have made use of Claude Sonnet 4.6 (Anthropic, 2026) to support several development and research activities.

1. In stage 2 to refine and improve prompts for the LLM models.

2. In the report to visualize figures like Fig.1.